Image Classification with Neural Networks: A Mathematical Approach to Flower Recognition

By: Emmalyn Holmquist
Advisor: Dr. Kuter

Overview

01 Motivation & Architecture of a Neural Network

02 Forward Pass

03 Backward Pass

04 Backpropagation

05 Gradient Descent

06 Application

Motivation

Image Classification

- Understand how images are classified using neural networks
- Use image classification to produce meaningful recommendations for e-commerce purposes
- End result: using neural networks to classify images of flowers as a stepping stone to image classification for recommendation systems

Image source: https://www.researchgate.net/figure/Figure-Artificial-Intelligence-Machine-Learning-and-Deep-Learning_fig3_340684782

What is a Neural Network?

- **Deep learning model** that we use to approximate an unknown function from data
  - Learns complex patterns
  - Automatically extracts useful features
  - Models highly non-linear relationships

- When do we use it over a regular machine learning model?
  - If data is very large and high dimensional
  - If it is difficult to manually design features
  - If relationships are nonlinear

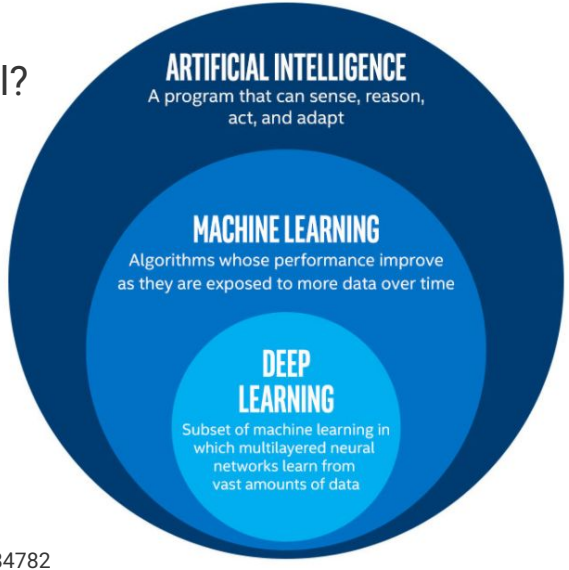
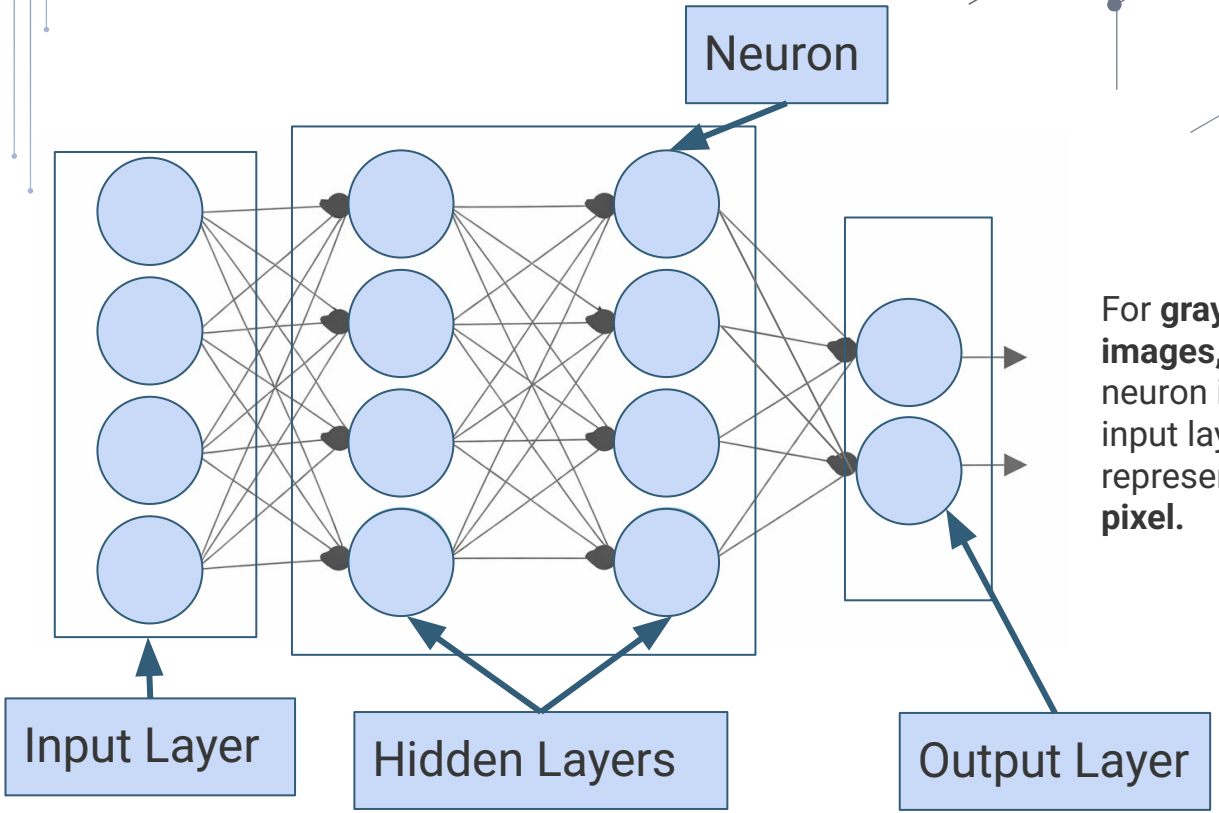


Image source: https://www.researchgate.net/figure/Figure-Artificial-Intelligence-Machine-Learning-and-Deep-Learning_fig3_340684782

Architecture of a Neural Network

Diagram of a Neural Network

Neural Networks can be used for **classification** or **regression** tasks, with image, text, speech, video, or other data, and are **highly customizable.**



For **grayscale images,** each neuron in the input layer may represent a **pixel.**

Image generated with AI

Practical Example

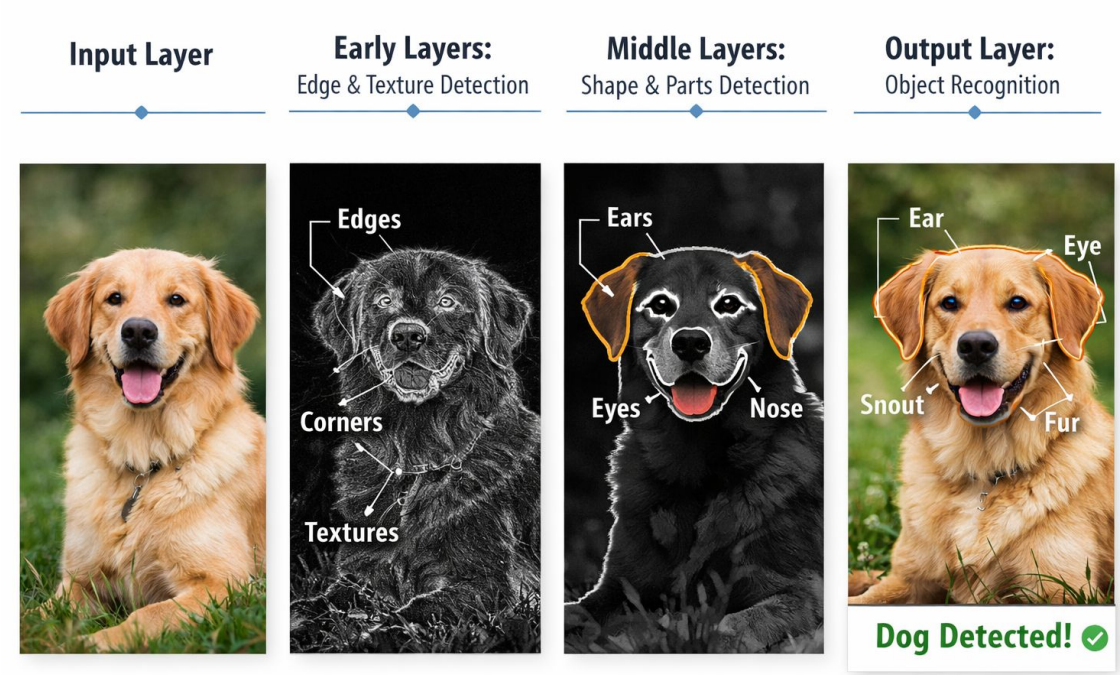


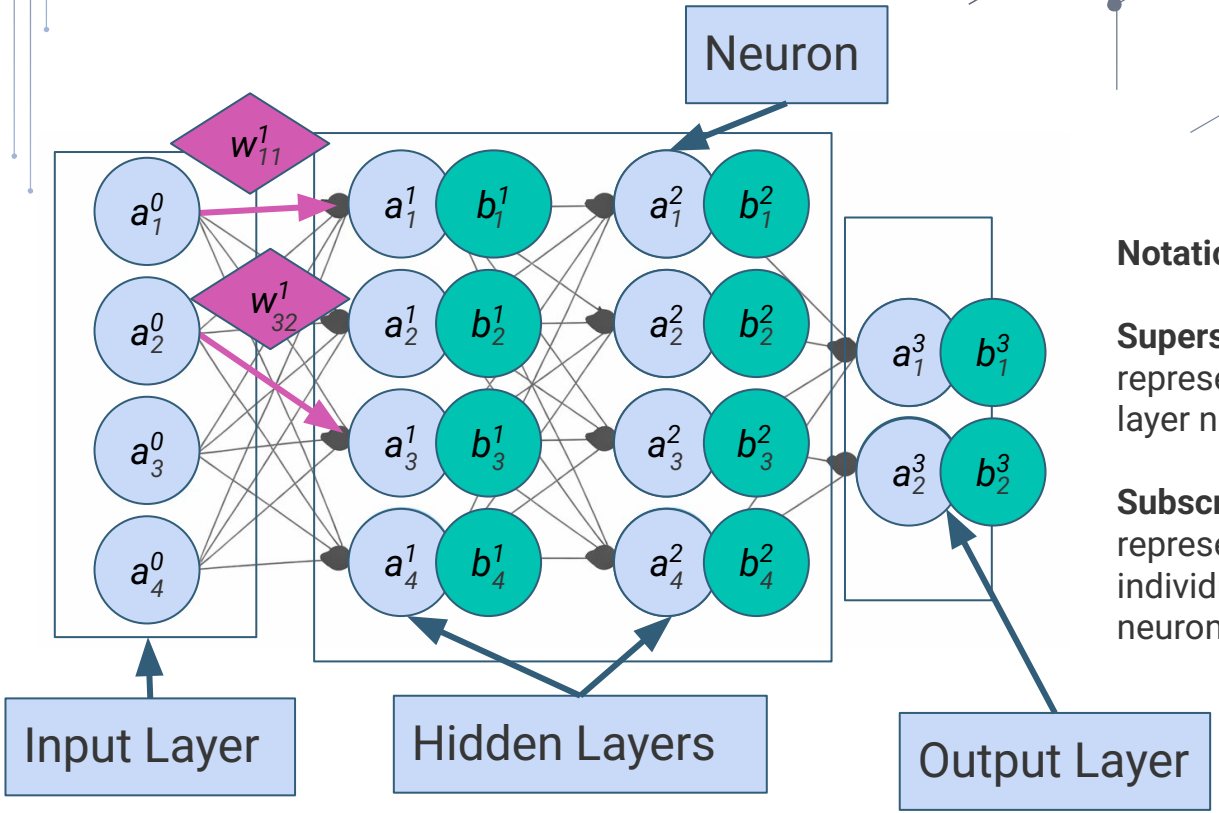
Image generated with AI

Diagram of a Neural Network

Each Neuron has an **activation** & **bias**.

Each connected edge represents a **weight.**

Note: a^0 represents the **input data**



Notation:

Superscripts represent the layer number

Subscripts represent the individual neuron

Image generated with AI

Weights, Biases, & Activation Functions

Weights

Give the strength of the connection between neurons, representing the importance of a particular feature

Biases

Provide a baseline/threshold for a neuron to activate

Activation Function

Introduce non-linearity to a network, allowing non-linearly seperable data to become linearly seperable

Weights & biases are our trainable parameters that we use to approximate the function. They are initialized randomly.

Forward Pass

Process of transforming the input data based on the current state of weights and biases (i.e., moving forward along the previous diagram to make a prediction).

Transformation

Every activation a^l_j of the jth neuron in the lth layer is related to activations in the (l-1)th layer by the equation:

Where σ is the elementwise activation function, w^l_{jk} is the weight from the neuron k to neuron *j*, a^{l-1}_k is the activation of the neuron from the previous layer corresponding to the weight, and b^l_j is the bias of neuron *j*.

$$a^l_j = \sigma\left(\sum_k w^l_{jk} a^{l-1}_k + b^l_j\right), \tag{23}$$

Which we can write in vector form as:

$$a^l = \sigma(w^l a^{l-1} + b^l). \tag{25}$$

Importance of σ

Adds non-linearity

This allows us to separate non-linearly separable data (such as images, speech, etc)

“Activates” Neurons

Decides which neurons to “fire” (pass information forward), helping network to learn meaningful features

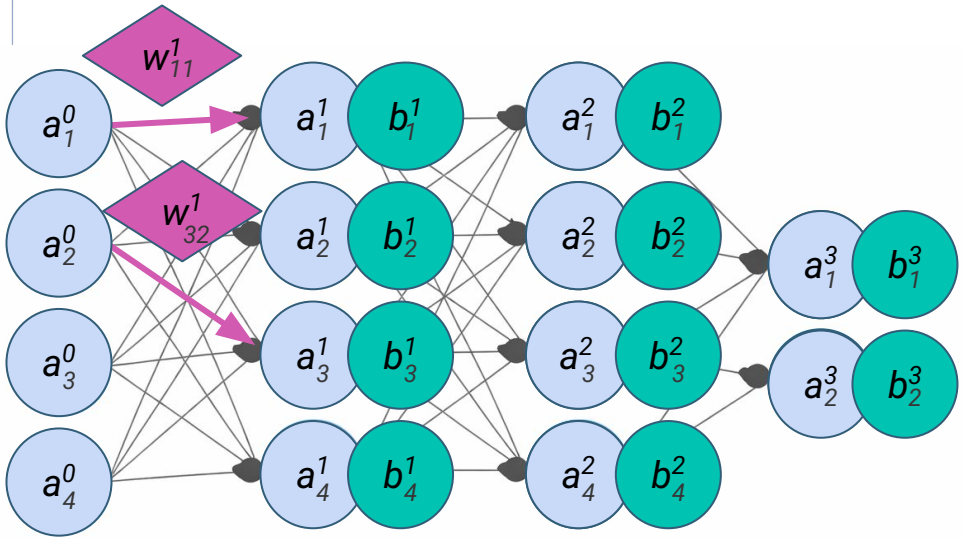
Enables Complex Decision Boundaries

Allows model to create curved decision boundaries (which are necessary for complex data)

Diagram of a Neural Network

Each Neuron has an **activation** & **bias**.

Each connected edge represents a **weight.**



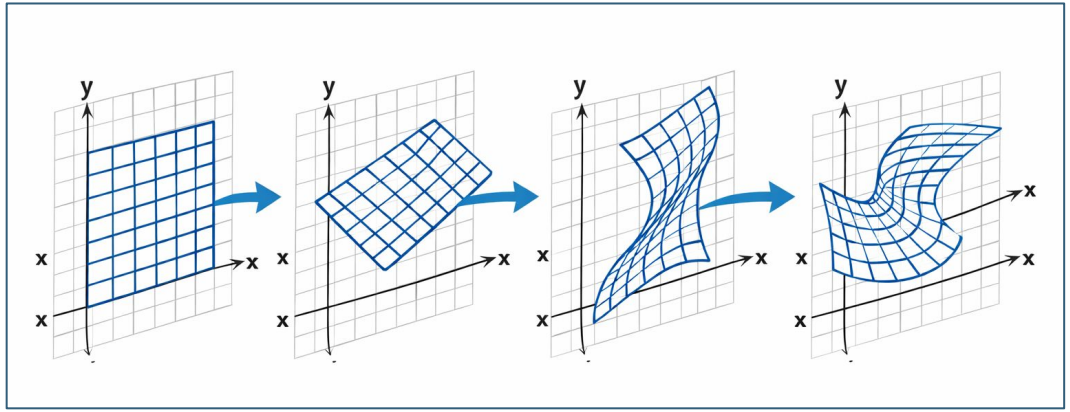
At each layer, we have a transformation of the input by the weight matrix and bias:

$\mathbf{a}^l = \sigma(\mathbf{W}^l\mathbf{a}^{l-1} + \mathbf{b}^l)$.

Image generated with AI

Visual

We can visualize our inputs being transformed by the layers as a plane being transformed on a graph.



$a^l = \sigma(w^l a^{l-1} + b^l)$ is just a matrix transformation!

Image generated with AI

Backward Pass

AKA where the actual learning happens!

What happens in the backward pass?

1. Compute the cost
   a. Using a **cost function**, compare the cost of the predicted to ground truth values
2. Calculate the gradients
   a. Determine how much each weight/bias contributed to the error
   b. We do this using **Backpropagation**
3. Adjust trainable parameters (weights & biases) to minimize the cost
   a. We adjust these slightly over many epochs using **Gradient Descent** (an optimizer)

But we do this for every image in the data set, over sometimes hundreds of **epochs (a full pass over the whole dataset!**

Important note: Learning is the process of updating the weights and bias to improve the outcome of the network/minimize the loss!

Step 1. Compute the Cost

End goal: Calculate the error of each layer to find how each weight/bias contributes to the cost of the predictions.

To do this, we first need to calculate the **cost** of our neural network (after the forward pass) by choosing a cost function and calculating the cost of the last layer.

We typically choose a cost function depending on the type of problem our network is trying to solve, and what our activation function is in the last layer.

Common examples for classification problems:

- Binary Classification: sigmoid activation in last layer -> binary cross entropy cost function
- Multi-Class Classification: softmax activation in last layer -> categorical cross entropy cost function

Backpropagation

Step 2. Backpropagation

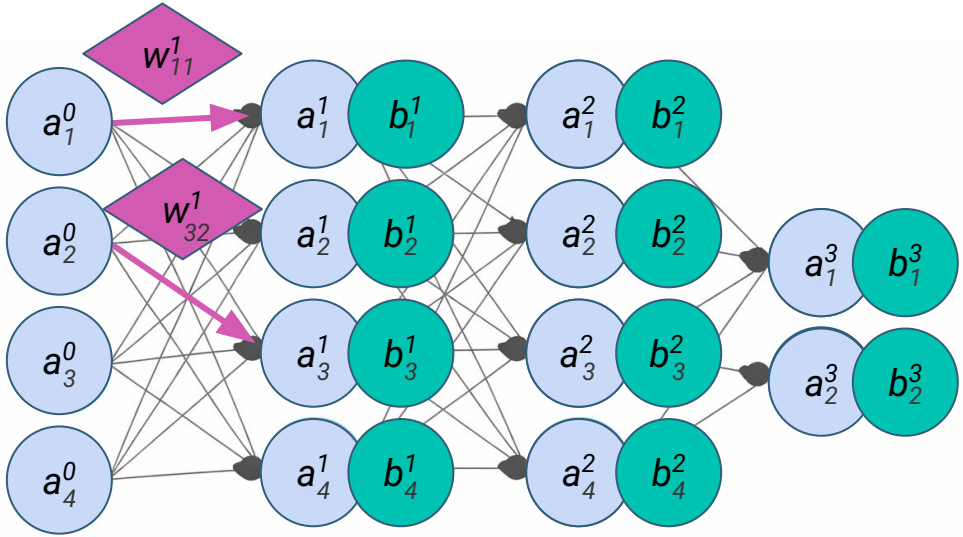
To calculate the error of the output, we need to calculate the **gradients** of the **cost function** to understand how **changing any one of the weights/biases** changes the **output** of the cost function (thus improving our predictions!).

Thus, we utilize the **chain rule** to **propagate backwards** through layers.

Recall: Diagram of a Neural Network

Intuitively, **each activation** is based on the activations, weights, and bias of the ***previous*** layer.

Thus, to find how the **current weight/bias** affects the output of the network, we need to determine how we affect the next layer in the network!



Loss $C = C(a^L)$,

Where L is the last layer.

Concept: Since our network consists of nested functions, taking the gradient of the last layer in our function depends on the result of other functions. The **chain rule** tells us how to compute the derivative of a function that **depends on** other functions!

Image generated with AI

4 Essential Equations for Backprop

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \tag{BP1}$$

$$\delta^l = \left((W^{l+1})^T \delta^{l+1}\right) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

Equation 1

$$\delta^L = \nabla_a C \odot \sigma'(z^L). \qquad \text{(BP1a)}$$

Purpose: gives the error in the output layer

- $\nabla_a C$ is the gradient of the cost w.r.t. the activation vector in the last layer
- $\odot$ is the Hadamard product (element-wise product of two vectors)
- $\sigma'(z^L)$ is how fast the activation function σ changes w.r.t. the input to the activation function in the last layer (or input to current activation)
- z = input to the activation function from the previous layer

Equation 2

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l), \qquad \text{(BP2)}$$

Purpose: gives us the error of the current layer l in terms of the next layer l + 1

- $(w^{l+1})^T$ is the transpose of the weight matrix for the l+1 layer
- δ^{l+1} is the error of the l + 1 layer
- $\odot$ is the Hadamard product (element-wise product of two vectors)
- $\sigma'(z^l)$ is how fast the activation function σ changes at the layer l w.r.t. the input to the activation at layer l (or input to current activation)

Equation 3

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l. \qquad \text{(BP3)}$$

Purpose: gives us an equation for the rate of change of the cost w.r.t. any bias in the network

- ∂b^l_j is the bias in layer l at neuron j
- δ^l_j is the error of the layer l at neuron *j*

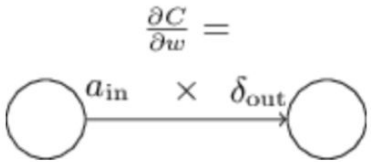
Equation 4

$$\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \delta^l_j. \qquad \text{(BP4)}$$

Purpose: gives us an equation for the rate of change of the cost w.r.t. any weight in the network

- ∂w^l_{jk} is the weight in layer l that goes to neuron j from neuron k
- a^{l-1}_k is the activation of the previous layer from neuron k
- δ^l_j is the error at layer l neuron j

Looking at just the two neurons the weight *w* is connected to, we can depict this as:



Example: Finding the Error of a Network with Sigmoid & BCE

Binary Cross Entropy:

$$C(y, \hat{y}) = -y \ln \hat{y} - (1 - y) \ln(1 - \hat{y})$$

Predictions as a function of Sigmoid Activation:

$$\hat{y} = \sigma\left(z^L\right) = \frac{1}{1 + e^{-z^L}}.$$

Complement of Predictions:

$$1 - \hat{y} = 1 - \frac{1}{1 + e^{-z^L}} = \frac{e^{-z^L}}{1 + e^{-z^L}}$$

Gradient Descent

Step 3. Gradient Descent

After we have calculated the **error** of the network (thus calculating the gradients at every layer and cost due to every weight and bias), we need to **update the weights and biases** based on our learned information in order to minimize the cost function.

This is done iteratively over many epochs by moving parameters in the direction that reduces the cost.

We can visualize minimizing the cost as if we are trying to descend to the bottom of *Cost Mountain:*

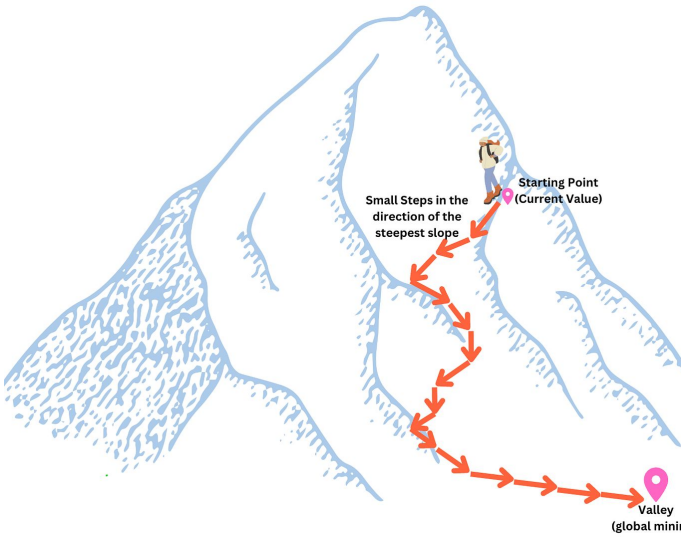


Image source: https://pub.towardsai.net/gradient-descent-and-the-melody-of-optimization-algorithms-244830ea2516

Gradient Descent Cont.

Recall: the gradient points in the direction of the steepest *increase*.

Thus, to **decrease cost,** we move the parameter in the **opposite direction** of the gradient.

For each parameter:

$w = w - \eta(\partial C/\partial w)$
$b = b - \eta(\partial C/\partial b)$

Where we update the weight and bias based on the error of the cost w.r.t. that parameter, and η is the learning rate.

*Note: $\partial C/\partial w$ and $\partial C/\partial b$ are the **average** cost.*

Improving Gradient Descent

Since calculating the gradients of the entire network over **every single input** for every epoch can get **computationally expensive,** thus we can introduce other methods to improve performance:

Mini-batch GD: Calculate $\partial C/\partial w$ *and* $\partial C/\partial b$ based on a small batch of the training data

Stochastic GD: Calculate $\partial C/\partial w$ *and* $\partial C/\partial b$ based on a single input from the training data

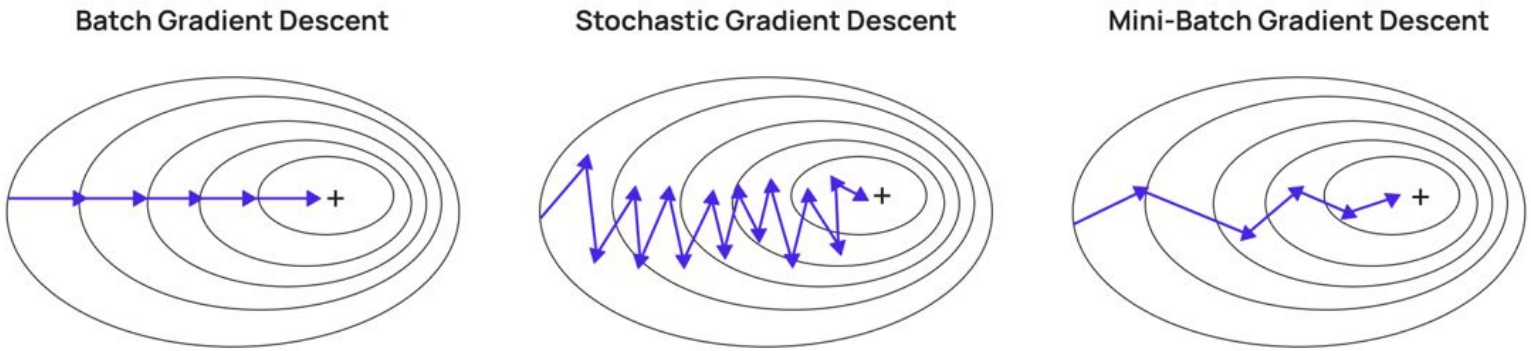


Image source: https://alwaysai.co/blog/what-is-gradient-descent

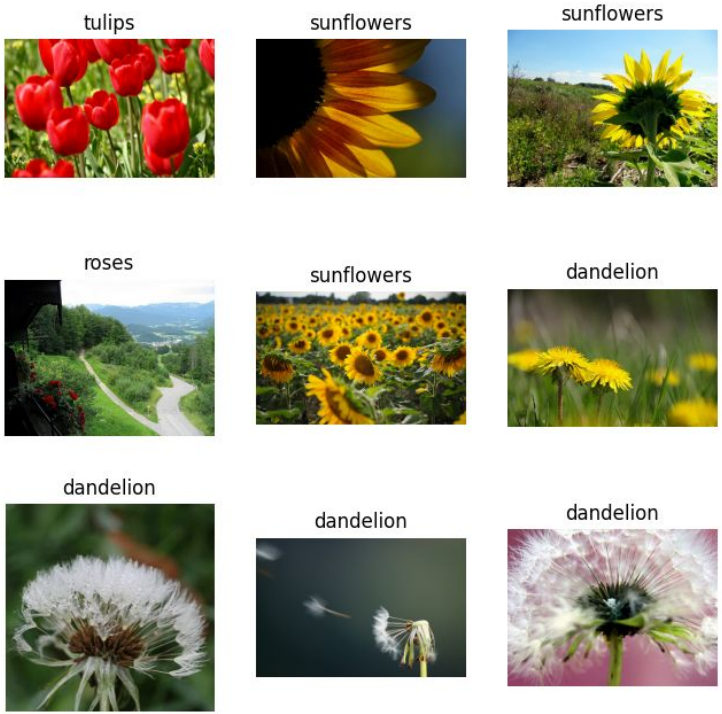
Application

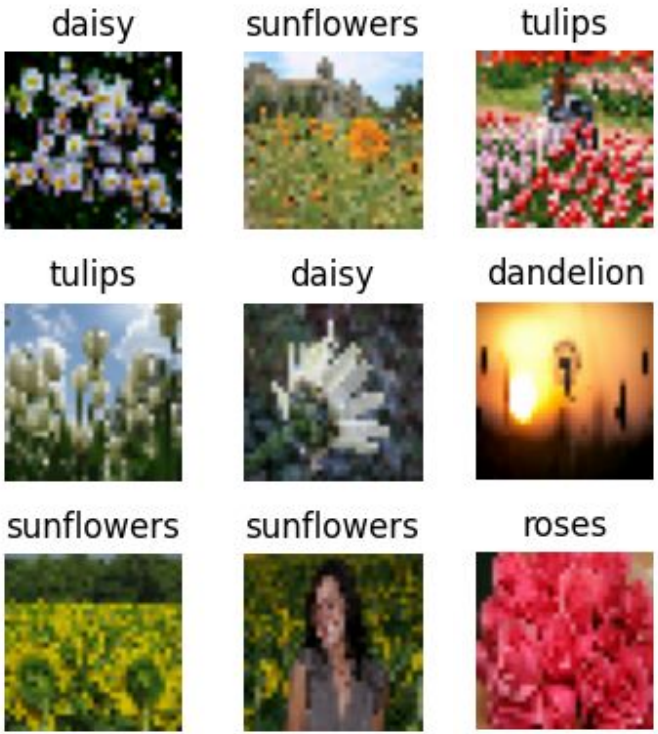
Setup

- **`tf_flowers`** dataset from **`TensorFlow`**
  - Data is loaded as **`tensors`** and converted to a flattened **`NumPy`** array

- Structure of network:
  - **Input Layer 0:** 3072 neurons (32px x 32px x 3 channels)
  - **Hidden Layer 1:** 128 neurons
  - **Hidden Layer 2:** 64 neurons
  - **Output Layer 3:** 5 neurons (5 flower classes - Daisy, Sunflowers, Tulips, Roses, Dandelion)

Pre-processing





Forward Pass

```python
# Forward Pass
def forward(X):

    # Layer 1 -> Input to Hidden
    # Linear transformation:
    # Z1 = XW1 + b1
    # X shape:  (batch_size, 307
    # W1 shape: (3072, 128)
    # Result:   (batch_size, 128
    Z1 = X @ W1 + b1

    # Apply ReLU activation: con
    # Shape remains (batch_size,
    A1 = relu(Z1)

    # Layer 2 -> Hidden Layer 1
    # Linear transformation:
    # Z2 = A1W2 + b2
    # A1 shape: (batch_size, 128
    # W2 shape: (128, 64)
    # Result:   (batch_size, 64)
    Z2 = A1 @ W2 + b2

    # Apply ReLU activation again. Adds non-linearity so model can learn complex patterns
    A2 = relu(Z2)
```

```python
    # Layer 3 -> Hidden Layer 2 to Output Layer
    # Linear transformation:
    # Z3 = A2W3 + b3
    # A2 shape: (batch_size, 64)
    # W3 shape: (64, 5)
    # Result:   (batch_size, 5)
    Z3 = A2 @ W3 + b3

    # Apply softmax activation. Converts raw scores (logits) into probabilities
    # Each row will sum to 1
    A3 = softmax(Z3)

    # Store intermediate values for later use (backpropagation). These will be needed to compute gradients
    cache = (Z1, A1, Z2, A2, Z3, A3)

    # A3 -> final predictions (probabilities)
    # cache -> saved values for training (backprop)
    return A3, cache
```

Backpropagation

```python
# Backpropagation
def backward(X, y, cache):

    # Unpack stored values from forward pass
    # These were saved to avoid recomputing them
    Z1, A1, Z2, A2, Z3, A3 = cache

    # Number of training examples
    m = X.shape[0]

    # Layer 3 -> output layer ---
    # Copy A3 so we don't modify original predictions
    # A3 = softmax output (probabilities)
    dZ3 = A3.copy()

    # Subtract 1 from the correct class probabilities
    # This is the gradient of softmax + cross-entropy loss
    # For each example i:
    #   dZ3[i, y[i]] -= 1
    dZ3[range(m), y] -= 1

    # Divide by m to average gradients over all samples
    dZ3 /= m

    # Compute gradient of weights for layer 3
    # A2.T shape: (64, m)
    # dZ3 shape:  (m, 5)
    # Result:     (64, 5)
    dW3 = A2.T @ dZ3

    # Compute gradient of biases for layer 3
    # Sum over all samples (axis=0 -> column-wise)
    # keepdims=True keeps shape as (1, 5)
    db3 = np.sum(dZ3, axis=0, keepdims=True)
```

```python
# Layer 2 ---
# Backpropagate gradient to previous layer
# W3.T shape: (5, 64)
# dZ3 shape:  (m, 5)
# Result:     (m, 64)
dA2 = dZ3 @ W3.T

# Apply derivative of ReLU
# Only keep gradients where Z2 > 0
# (ReLU blocks negative values)
dZ2 = dA2 * (Z2 > 0)

# Gradient of weights for layer 2
# A1.T shape: (128, m)
# dZ2 shape:  (m, 64)
# Result:     (128, 64)
dW2 = A1.T @ dZ2

# Gradient of biases for layer 2
db2 = np.sum(dZ2, axis=0, keepdims=True)


# Layer 1 ---
# Backpropagate gradient further back
# W2.T shape: (64, 128)
# dZ2 shape:  (m, 64)
# Result:     (m, 128)
dA1 = dZ2 @ W2.T

# Apply derivative of ReLU again
dZ1 = dA1 * (Z1 > 0)

# Gradient of weights for layer 1
# X.T shape:  (3072, m)
# dZ1 shape: (m, 128)
# Result:    (3072, 128)
dW1 = X.T @ dZ1

# Gradient of biases for layer 1
db1 = np.sum(dZ1, axis=0, keepdims=True)

# Return all gradients - these will be used to update weights during training
return dW1, db1, dW2, db2, dW3, db3
```

Gradient Descent

```python
# Function to create mini-batches
def get_batches(X, y, batch_size=32):

    # Create a random permutation of indices from 0 to len(X)-1
    # This shuffles the dataset so batches are different each epoch
    # Prevents the model from learning patterns based on data order
    indices = np.random.permutation(len(X))

    # Loop over the dataset in steps of batch_size
    # range(start, stop, step)
    # Example: range(0, 1000, 32)
    for i in range(0, len(X), batch_size):

        # Select a slice of shuffled indices for this batch
        # i:i+batch_size gives a chunk of indices
        batch_idx = indices[i:i+batch_size]

        # Yield (return) the batch of inputs and labels
        # X[batch_idx] -> selects those rows from X
        # y[batch_idx] -> selects corresponding labels
        # 'yield' makes this a generator (memory efficient)
        yield X[batch_idx], y[batch_idx]
```

```python
# Function to update parameters using gradient descent
def update_params(dW1, db1, dW2, db2, dW3, db3, lr):

    # Declare these variables as global so we can modify them
    # These are the model parameters defined earlier
    global W1, b1, W2, b2, W3, b3

    # Update Layer 1 parameters ---
    # Update weights using gradient descent rule:
    # W = W - learning_rate * gradient
    W1 -= lr * dW1
    # Update biases similarly
    b1 -= lr * db1

    # Update Layer 2 parameters ---
    # Adjust weights for layer 2
    W2 -= lr * dW2
    # Adjust biases for layer 2
    b2 -= lr * db2

    # Update Layer 3 (output layer) parameters ---
    # Update weights for output layer
    W3 -= lr * dW3
    # Update biases for output layer
    b3 -= lr * db3
```

Training Loop

```
Epoch 0, Loss: 1.3710
Epoch 10, Loss: 1.0038
Epoch 20, Loss: 1.0081
Epoch 30, Loss: 0.6216
Epoch 40, Loss: 0.4514
```

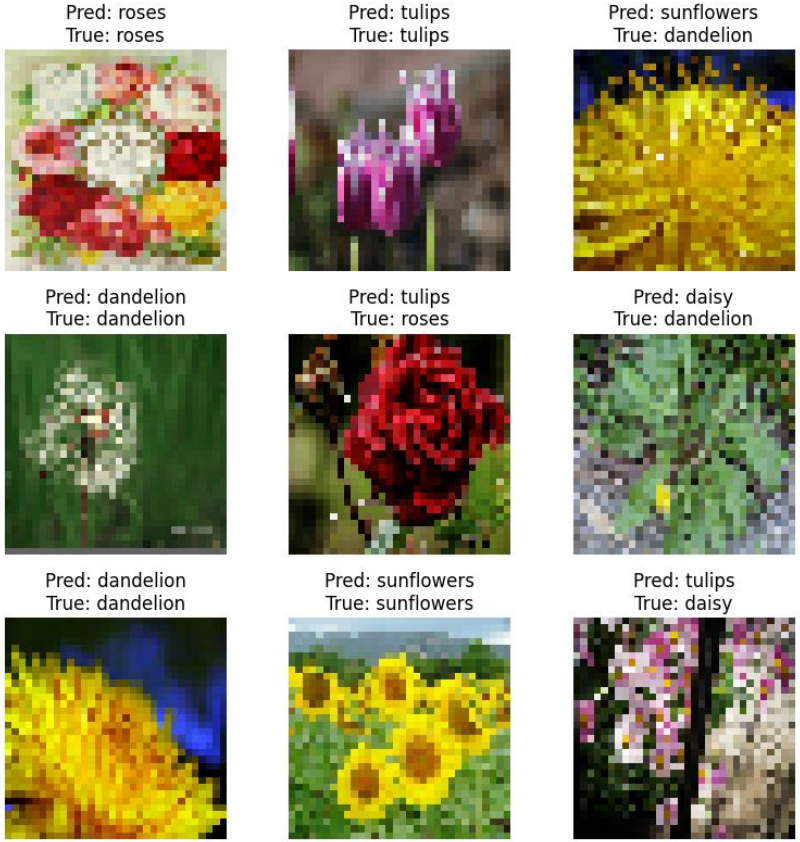
```python
# Loop over the dataset multiple times (epochs)
# One epoch = one full pass through the entire dataset
for epoch in range(50):

    # Iterate over mini-batches of data
    # get_batches() returns small chunks of (X, y)
    for X_batch, y_batch in get_batches(X_train, y_train):

        # Forward pass ---
        # Compute predictions (A3) and store intermediate values (cache)
        A3, cache = forward(X_batch)

        # Backward pass ---
        # Compute gradients of weights and biases using backpropagation
        # These gradients tell us how to adjust parameters
        dW1, db1, dW2, db2, dW3, db3 = backward(X_batch, y_batch, cache)

        # Parameter update ---
        # Update weights and biases using gradient descent
        # lr=0.01 is the learning rate (step size)
        update_params(dW1, db1, dW2, db2, dW3, db3, lr=0.01)

    # Monitor training progress ---
    # Every 10 epochs, print the loss
    if epoch % 10 == 0:

        # Calculate loss over the entire dataset to report
        A3_full, _ = forward(X_train)
        # Compute loss every 10th epoch - for testing purposes, not training
        loss = compute_loss(A3_full, y_train)

        # Print current epoch number and loss value
        # :.4f formats the loss to 4 decimal places
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

# Calculate loss over the entire dataset to report
A3_full, _ = forward(X_train)

# Compute loss for final epoch
loss = compute_loss(A3_full, y_train)
```

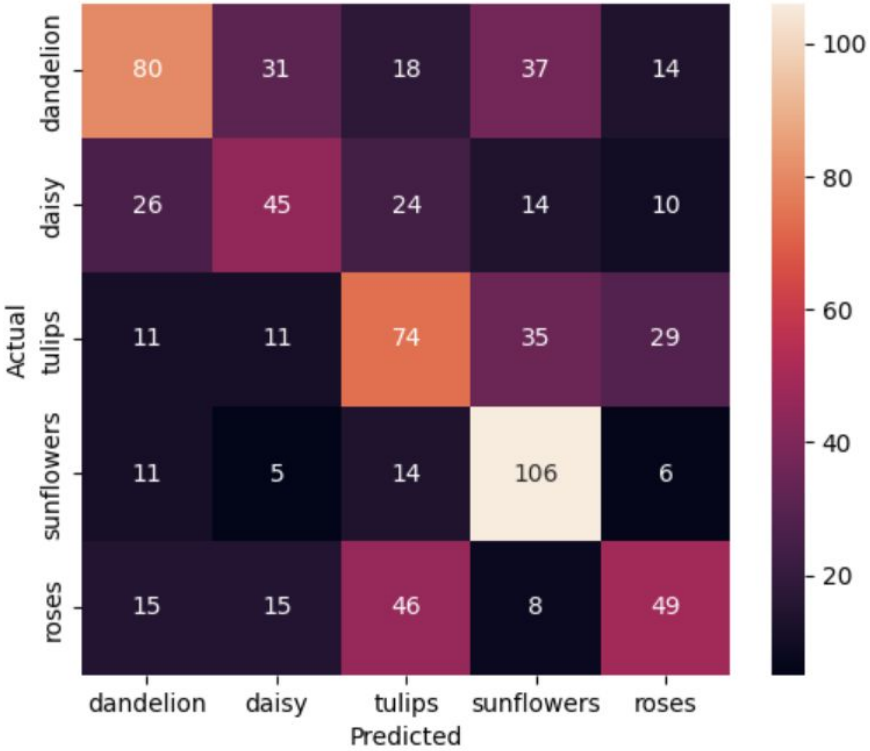
Predict

```python
# Function to make predictions
def predict(X):

    # Run a forward pass through the network
    # A3 contains the output probabilities for each class
    # Shape: (num_samples, num_classes)
    # We ignore the cache (_) because we don't need it for prediction
    A3, _ = forward(X)

    # Select the index of the highest probability for each sample
    # axis=1 -> operate across columns (classes)
    # For each row, pick the class with the highest probability
    # Result shape: (num_samples,)
    return np.argmax(A3, axis=1)

# Use the model to make predictions on dataset X_test
# preds will contain predicted class labels (integers)
preds = predict(X_test)

# Compute accuracy
# Compare predictions to true labels (y_test). (preds == y_test) creates a boolean array: True where prediction is correct, False otherwise
# np.mean(...) computes fraction of correct predictions: True = 1, False = 0
# So mean = (# correct) / (total samples)
# Multiply by 100 -> convert to percentage
accuracy = np.mean(preds == y_test) * 100

print(f"Accuracy: {round(accuracy, 2)}%")
```

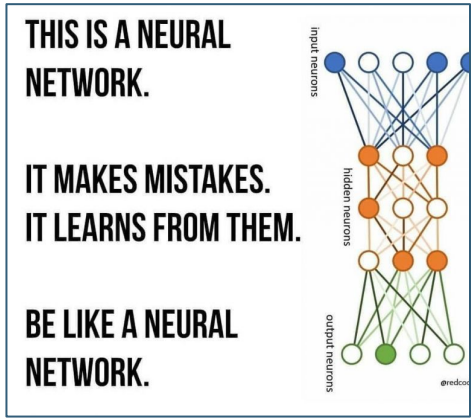
```
Accuracy: 44.28%
```

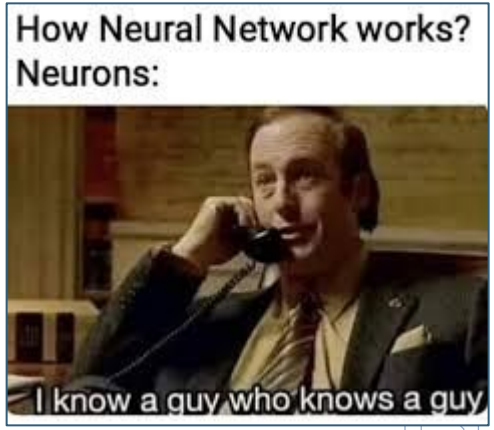
Visualize Predictions

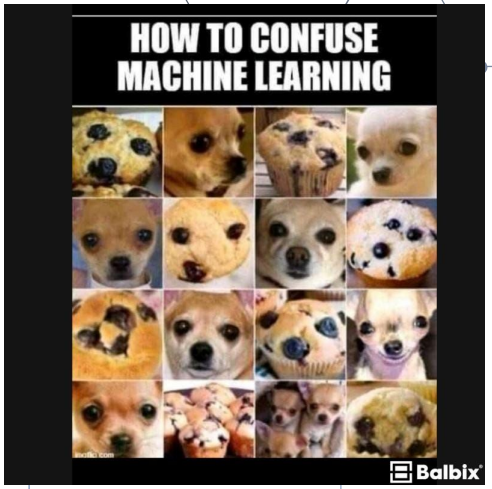




Thanks !







CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**

Sources

[1] Michael A. Nielsen, "Neural Networks and Deep Learning", *Neural Networks and Deep Learning*, Determination Press, 2015, http://neuralnetworksanddeeplearning.com/.

[2] François Chollet and Matthew Watson, "Deep Learning with Python, Third Edition", *Deep Learning with Python*, Manning Publications, 2025, ISBN 978-1-63343-658-9, https://www.manning.com/books/deep-learning-with-python-third-edition.

[3] Grant Sanderson, "Neural Networks (Deep Learning) — 3Blue1Brown Playlist", *YouTube*, 3Blue1Brown channel, 2017, https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi.